The Theory of Computing

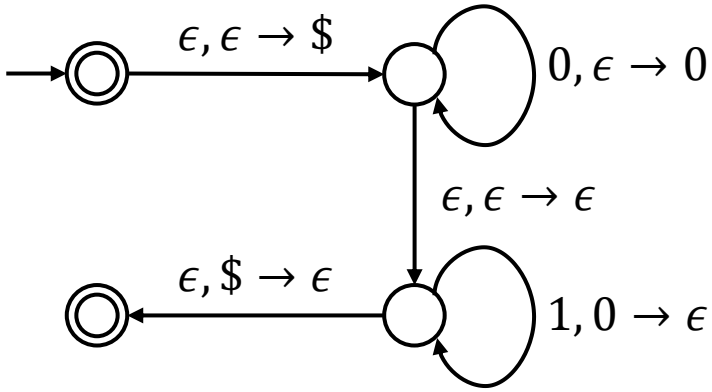
Pushdown Automata

A pushdown automaton is a non-deterministic finite automaton augmented with a stack for memory.

In a pushdown automaton, during a transition a symbol can be pushed or popped from the stack.

$$\sigma, <\text{pop}> \rightarrow <\text{push}>$$

$$L_1 = \{0^n 1^n | n \geq 0\}$$



Definition: A pushdown automaton (PDA) is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where:

1. Q is a finite set of *states*,
2. Σ is a finite *input alphabet*,
3. Γ is a finite *stack alphabet*,
4. $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to P(Q \times \Gamma_\epsilon)$ is the *transition function*,
5. $q_0 \in Q$ is the *start state*, and
6. $F \subseteq Q$ is the *set of accept states*.

Recall that $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$.

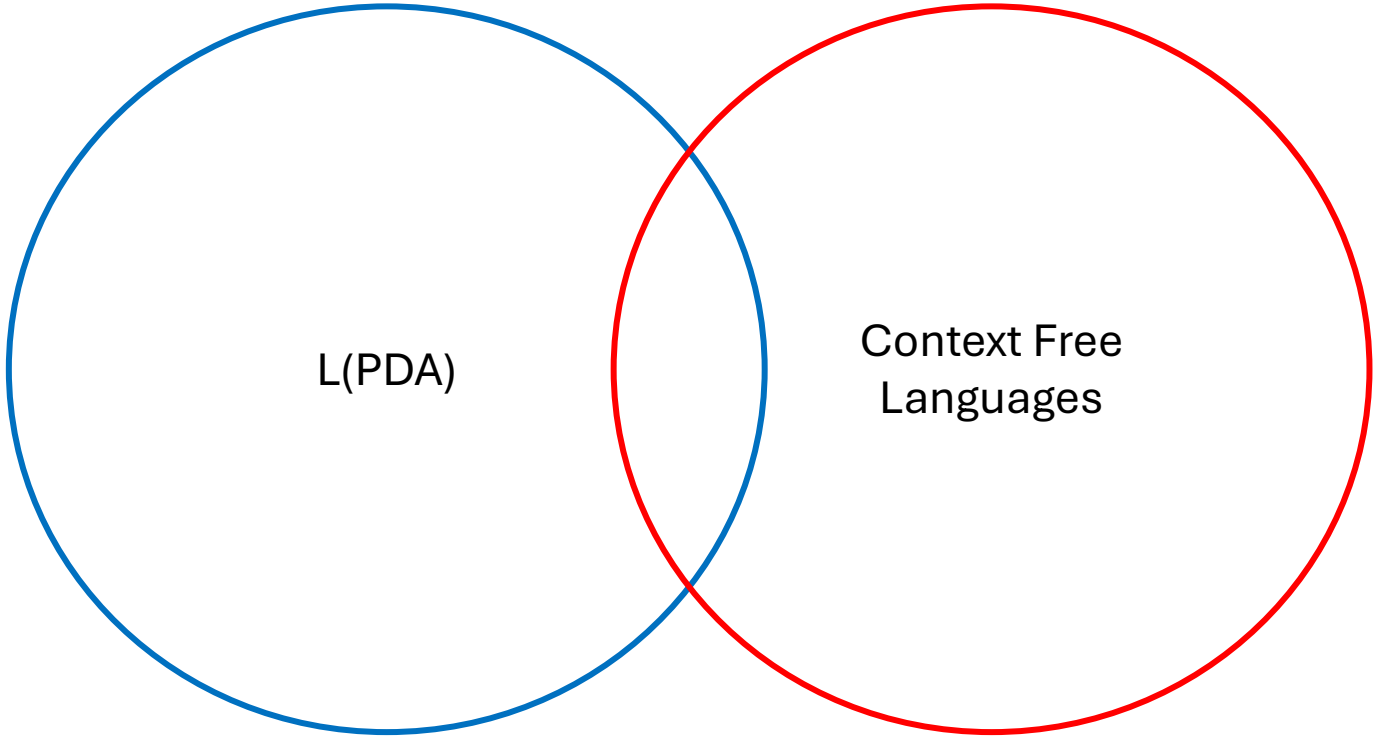
A pushdown automaton has no way to detect an empty stack. Often, we use the special symbol, $, to mark the bottom of the stack.

Exercises

$L_0 = \{w | w \text{ is a palindrome}\}$
$L_1 = \{0^n 1^m 0^m 1^n | n, m \geq 0\}$
$L_2 = \{w | \text{ the middle symbol of } w \text{ is a } 0\}$
$L_3 = \{w | \ w \text{ contains equal numbers of } 0s \text{ and } 1s\}$
$L_4 = \{0^i 1^j 2^k | i = j \text{ or } j = k\}$
$L_5 = \{w | w \text{ has twice as many } 0s \text{ as } 1s\}$
$L_6 = \{w\#x | w^R \text{ is a substring of } x\}$
$L_7 = \{w | w \text{ contains a 1 after its midpoint}\}$

The Theory of Computing

Pushdown Automata and Context Free Languages



What is the relationship between the languages recognized by push-down automata and the context free languages?

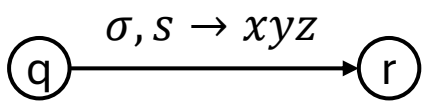
Spoilers: They are the same.

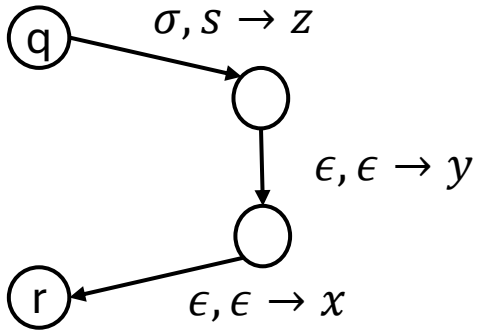
Theorem: A language is context free if and only if some pushdown automaton recognizes it.

Lemma: If a language is context free, then some pushdown automaton recognizes it.

Lemma: If a pushdown automaton recognizes some language, then it is context free.

We will use a shorthand that allows us to push more than one symbol to the stack at a time.





Lemma: If a language is context free, then some pushdown automaton recognizes it.

Proof idea: We will create a push-down automaton from a context free grammar with three primary states as follows:

This machine non-deterministically generates every possible string on the stack while processing no input.

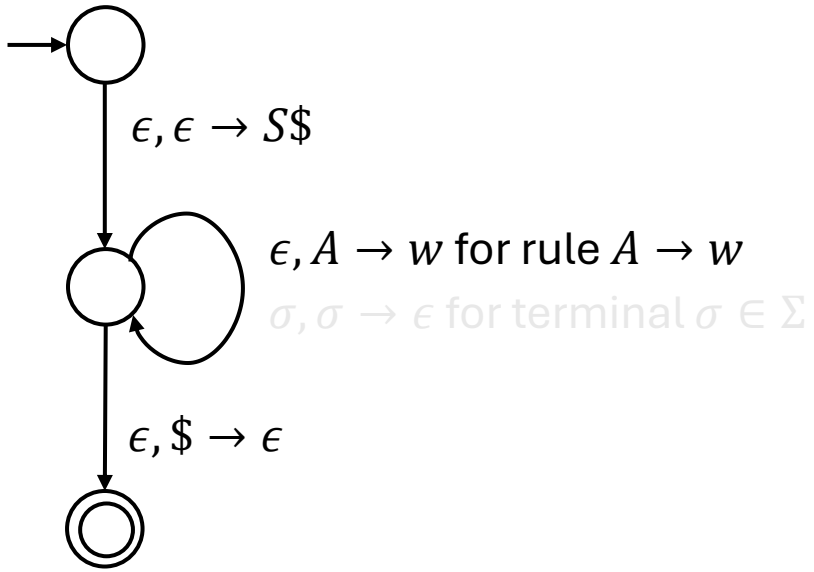
Then for each input character, the machine must also pop a character from the stack.

As the accept state can only be reached with an empty stack, any accepting computation corresponds to a string generated by the grammar.

So, this machine accepts a string if and only if it was generated by G.

Lemma: If a language is context free, then some pushdown automaton recognizes it.

Proof idea: We will create a push-down automaton from a context free grammar with three primary states as follows:



This machine non-deterministically generates every possible string on the stack while processing no input.

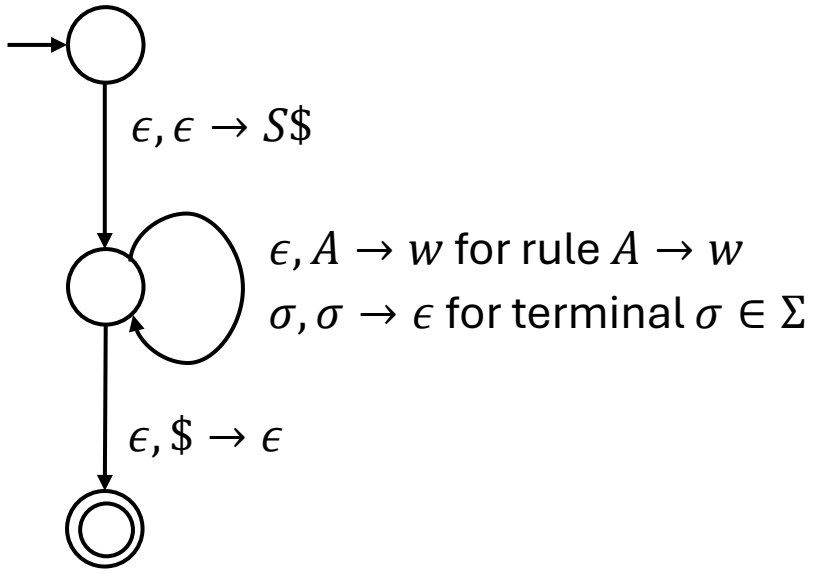
Then for each input character, the machine must also pop a character from the stack.

As the accept state can only be reached with an empty stack, any accepting computation corresponds to a string generated by the grammar.

So, this machine accepts a string if and only if it was generated by G.

Lemma: If a language is context free, then some pushdown automaton recognizes it.

Proof idea: We will create a push-down automaton from a context free grammar with three primary states as follows:



This machine non-deterministically generates every possible string on the stack while processing no input.

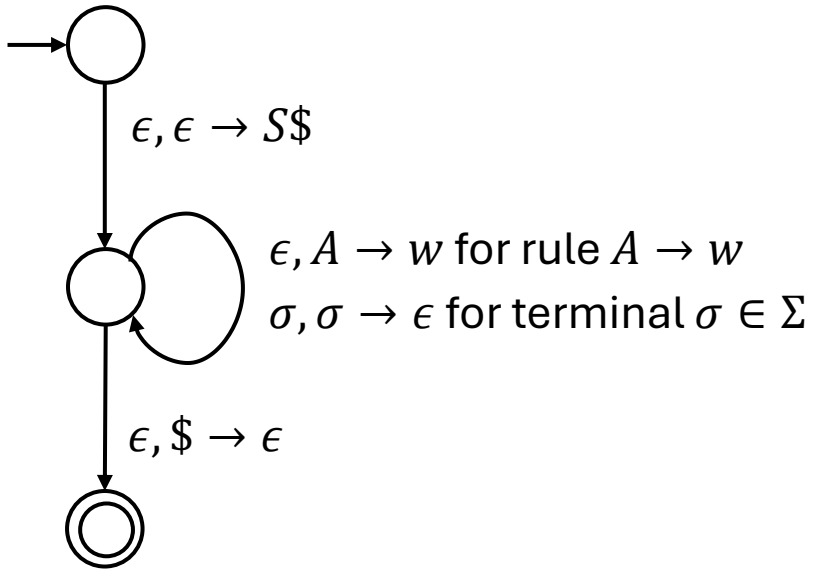
Then for each input character, the machine must also pop a character from the stack.

As the accept state can only be reached with an empty stack, any accepting computation corresponds to a string generated by the grammar.

So, this machine accepts a string if and only if it was generated by G.

Lemma: If a language is context free, then some pushdown automaton recognizes it.

Proof idea: We will create a push-down automaton from a context free grammar with three primary states as follows:



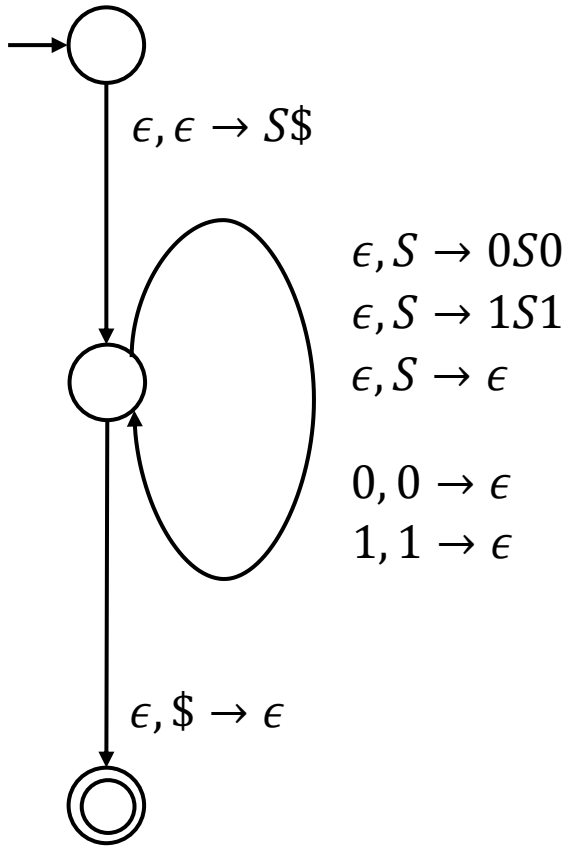
This machine non-deterministically generates every possible string on the stack while processing no input.

Then for each input character, the machine must also pop a character from the stack.

As the accept state can only be reached with an empty stack, any accepting computation corresponds to a string generated by the grammar.

So, this machine accepts a string if and only if it was generated by G.

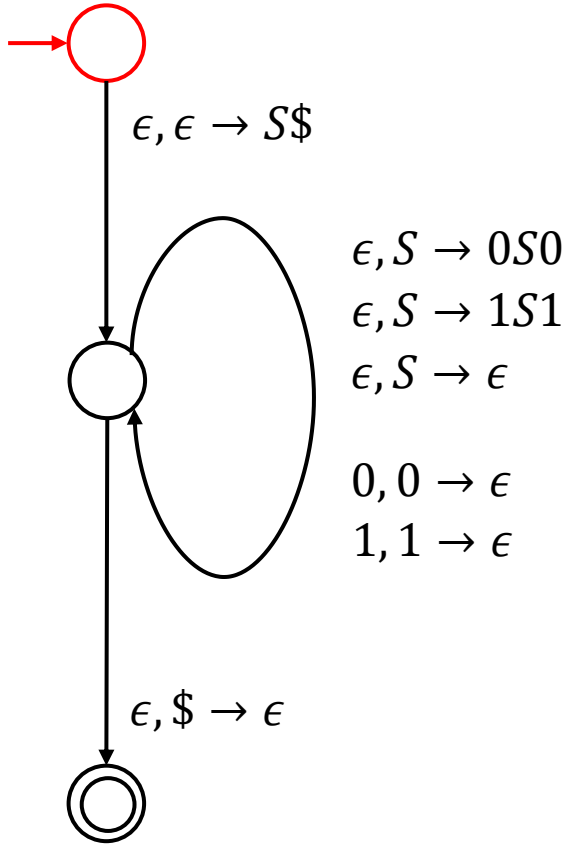
$S \rightarrow 0S0 \mid 1S1 \mid \epsilon$



$w = 0110$

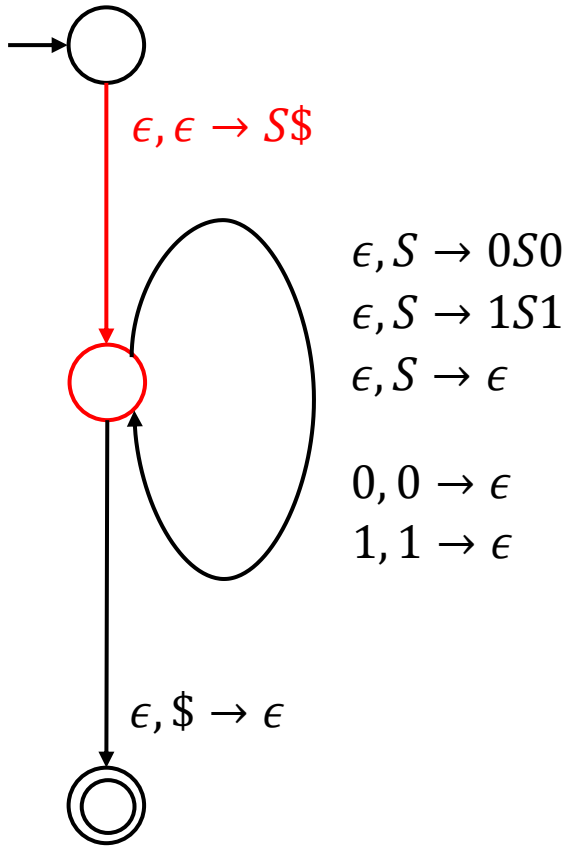
Stack Contents

ϵ



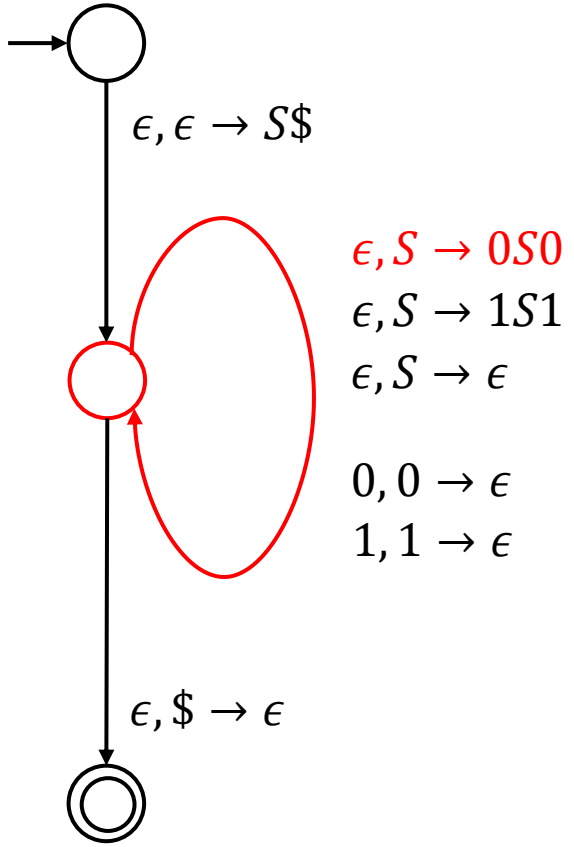
$w = 0110$

Stack Contents
$S\$$



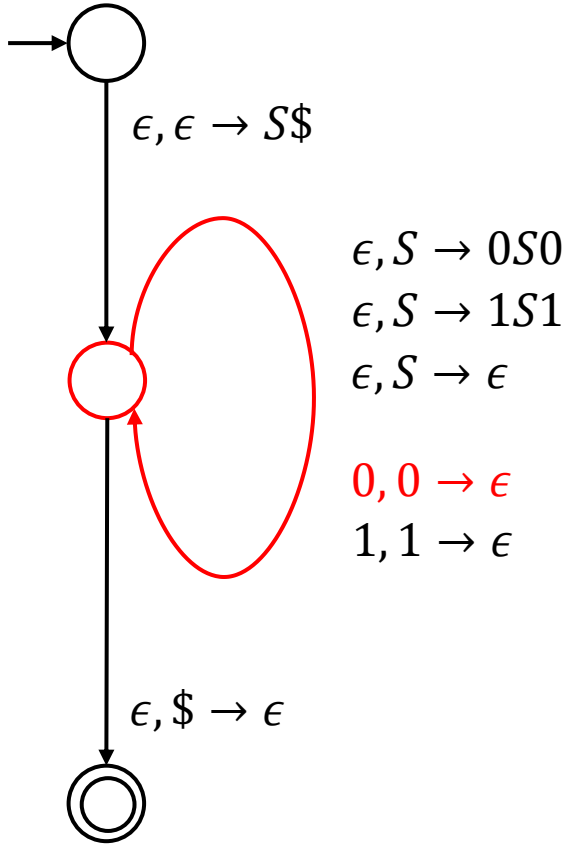
$w = 0110$

Stack Contents
$0S0\$$



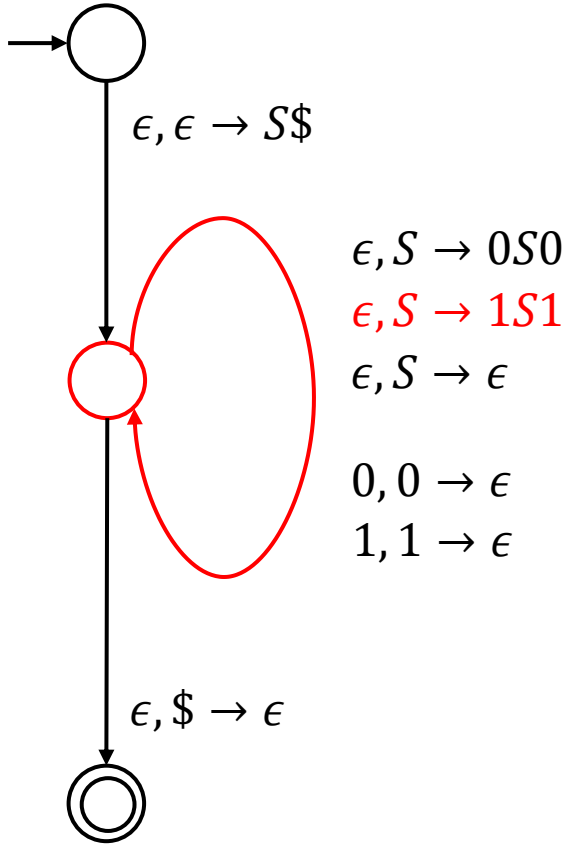
$w = 0110$

Stack Contents
$S0\$$



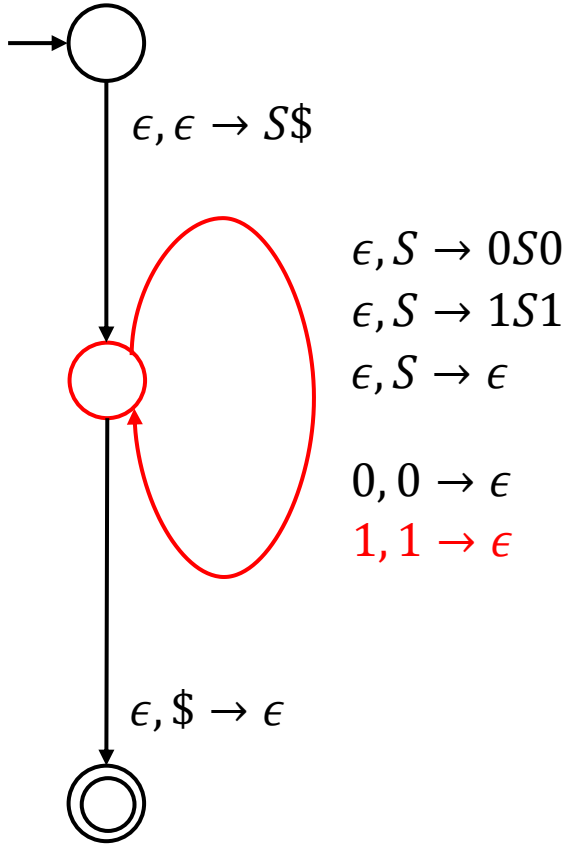
$w = 0110$

Stack Contents
$1S10\$$



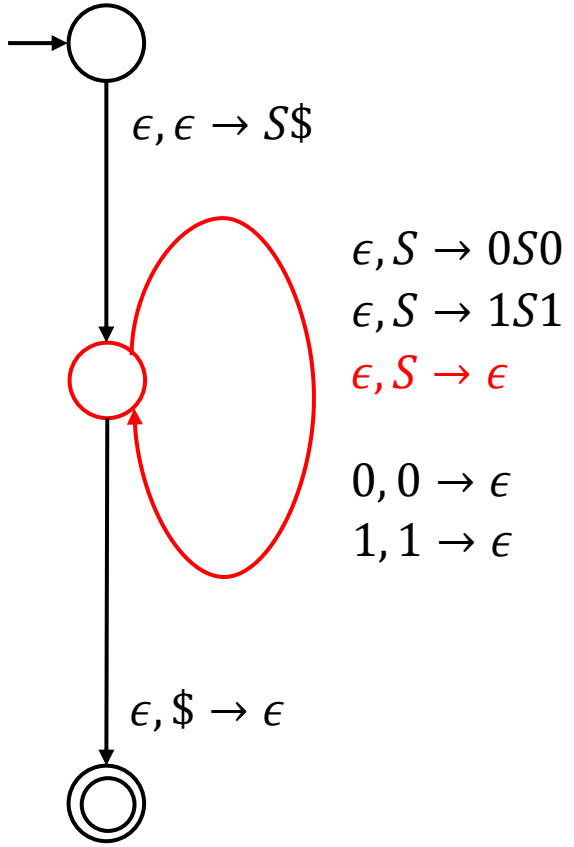
$w = 0110$

Stack Contents
$S10\$$



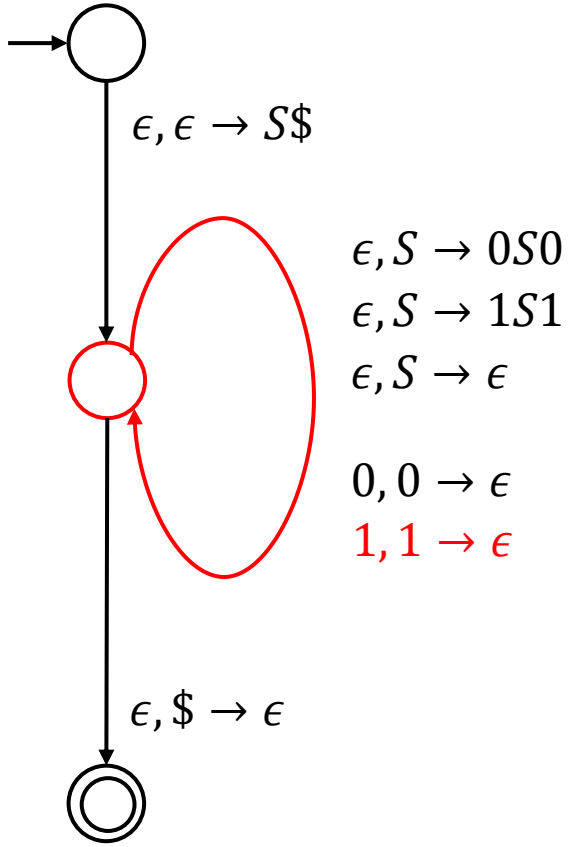
$w = 0110$

Stack Contents
$10\$$



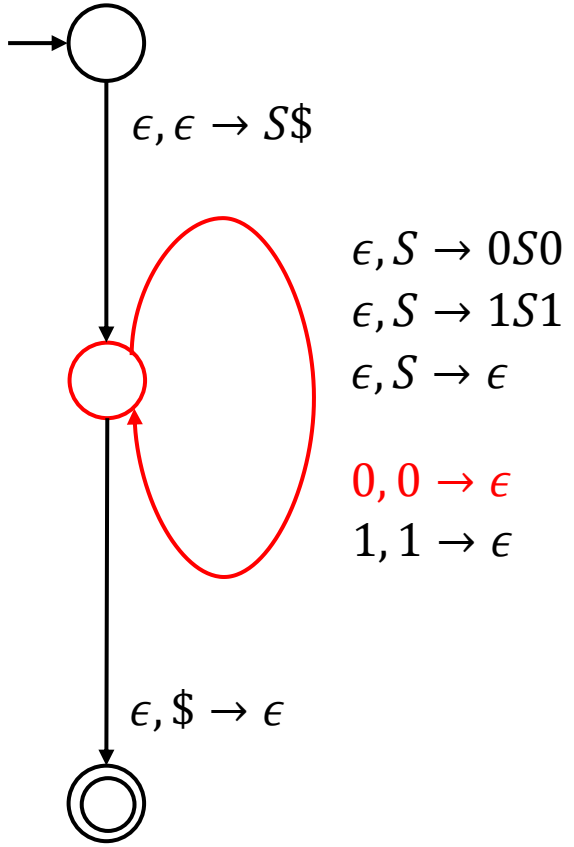
$w = 0110$

Stack Contents
$0\$$



$w = 0110$

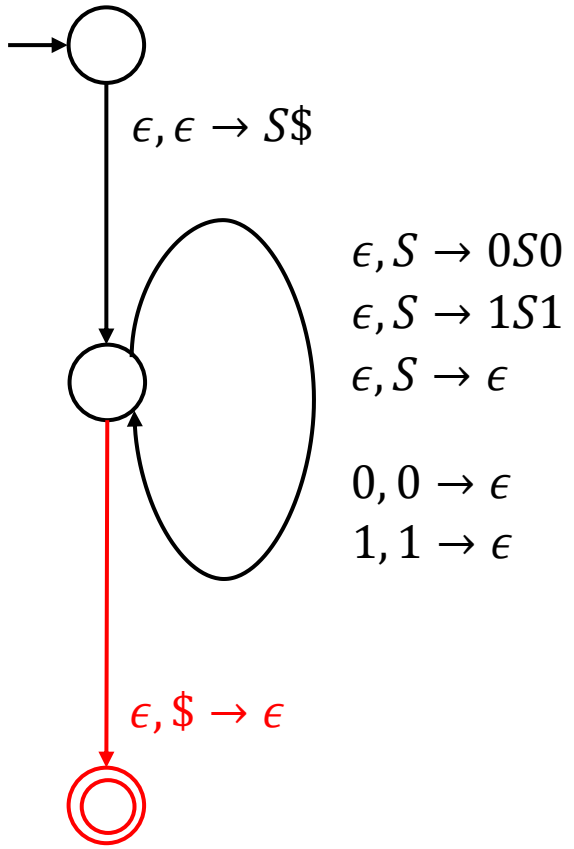
Stack Contents
$



$w = 0110$

Stack Contents

ϵ



The Theory of Computing

Pushdown Automata and Context Free Languages

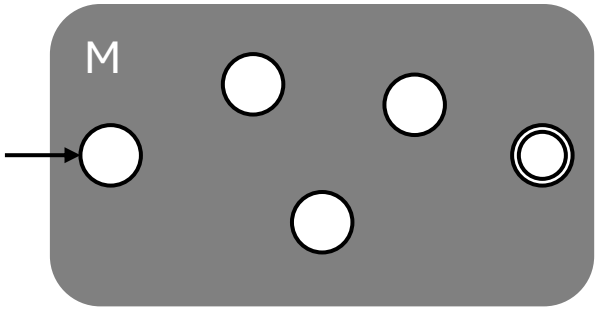
Theorem: A language is context free if and only if some pushdown automaton recognizes it.

Lemma: If a language is context free, then some pushdown automaton recognizes it.

Lemma: If a pushdown automaton recognizes some language, then it is context free.

Lemma: If a pushdown automaton recognizes some language, then it is generated by a context free grammar.

Proof idea: We will create a context free grammar from a push-down automaton with a single accept state and for which each stack operation is either a push or a pop (but not both).



The start rule of our grammar represents a path from the start state to the single accept state.

Other rules in the grammar will represent paths from one state, p, to another, q, in the automaton.

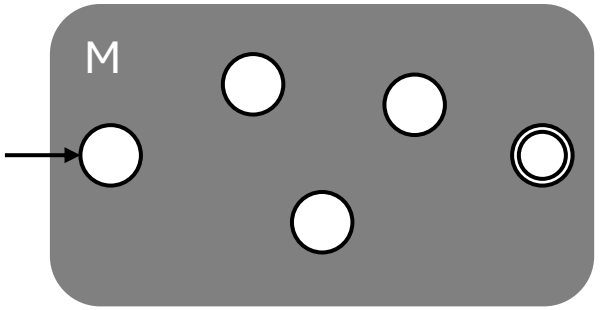
$$A_{pq}$$

Rules are created by matching symbols that are pushed and popped from the stack.

To ensure we create rules for all paths, we create more rules than we will need.

Lemma: If a pushdown automaton recognizes some language, then it is generated by a context free grammar.

Proof idea: We will create a context free grammar from a push-down automaton with a single accept state and for which each stack operation is either a push or a pop (but not both).



The start rule of our grammar represents a path from the start state to the single accept state.

Other rules in the grammar will represent paths from one state, p, to another, q, in the automaton.

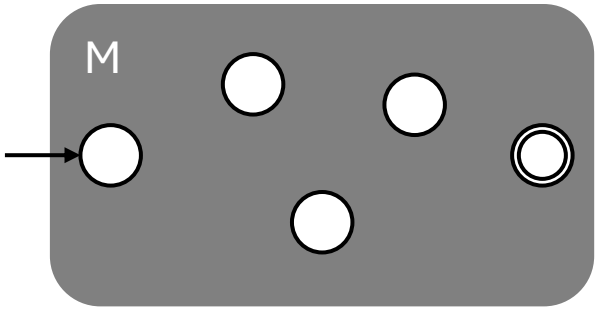
$$A_{pq}$$

Rules are created by matching symbols that are pushed and popped from the stack.

To ensure we create rules for all paths, we create more rules than we will need.

Lemma: If a pushdown automaton recognizes some language, then it is generated by a context free grammar.

Proof idea: We will create a context free grammar from a push-down automaton with a single accept state and for which each stack operation is either a push or a pop (but not both).



The start rule of our grammar represents a path from the start state to the single accept state.

Other rules in the grammar will represent paths from one state, p, to another, q, in the automaton.

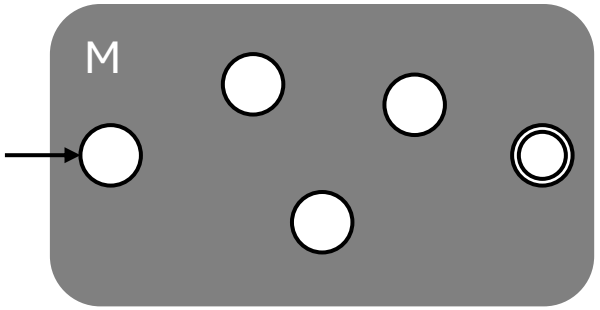
$$A_{pq}$$

Rules are created by matching symbols that are pushed and popped from the stack.

To ensure we create rules for all paths, we create more rules than we will need.

Lemma: If a pushdown automaton recognizes some language, then it is generated by a context free grammar.

Proof idea: We will create a context free grammar from a push-down automaton with a single accept state and for which each stack operation is either a push or a pop (but not both).



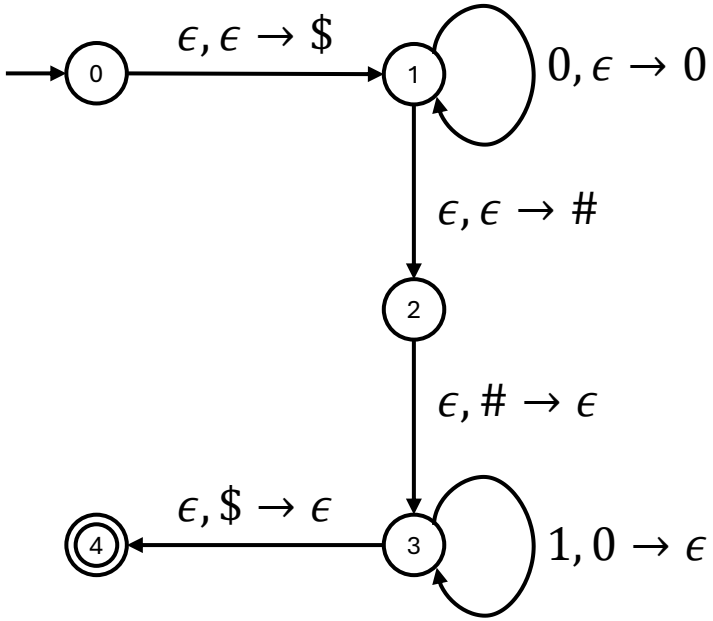
The start rule of our grammar represents a path from the start state to the single accept state.

Other rules in the grammar will represent paths from one state, p, to another, q, in the automaton.

$$A_{pq}$$

Rules are created by matching symbols that are pushed and popped from the stack.

To ensure we create rules for all paths, we create more rules than we will need.



$S \to A_{04}$

A_{00}

A_{01}

A_{02}

A_{03}

A_{04}

A_{11}

A_{12}

A_{13}

A_{14}

A_{22}

A_{23}

A_{24}

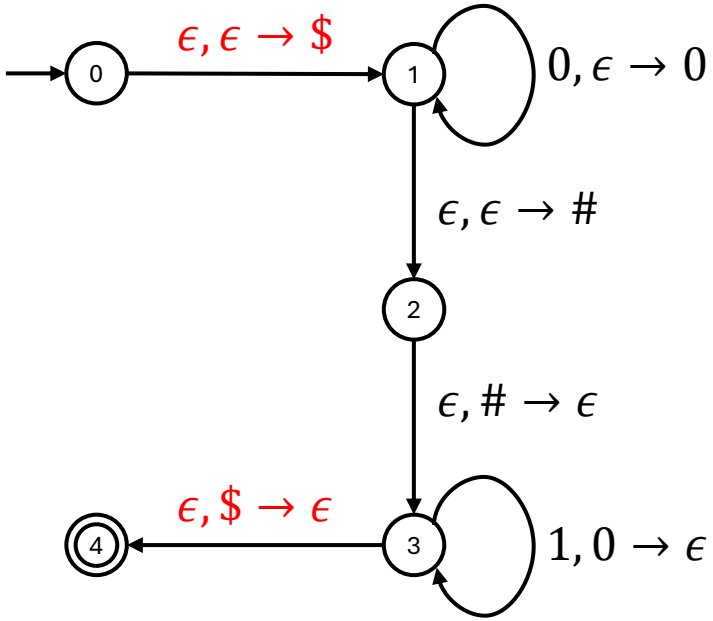
A_{33}

A_{34}

A_{44}



For each $p, q, r, s \in Q, u \in \Gamma$, and $a, b \in \Sigma_\epsilon$, if $\delta(p, a, \epsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ϵ) add the rule $A_{pq} \to aA_{rs}b$ to G.



$S \to A_{04}$

A_{00}

A_{01}

A_{02}

A_{03}

$A_{04} \to A_{13}$

A_{11}

A_{12}

A_{13}

A_{14}

A_{22}

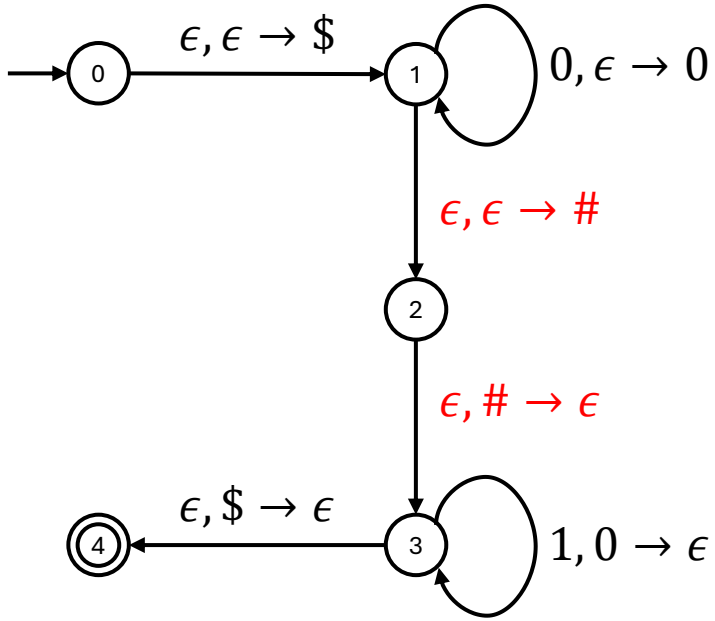
A_{23}

A_{24}

A_{33}

A_{34}

A_{44}



$S \to A_{04}$

A_{00}

A_{01}

A_{02}

A_{03}

$A_{04} \to A_{13}$

A_{11}

A_{12}

$A_{13} \to A_{22}$

A_{14}

A_{22}

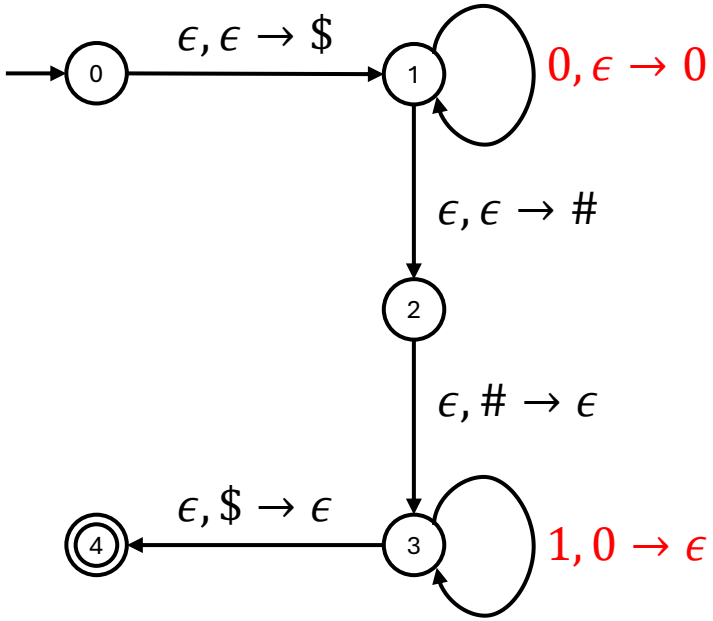
A_{23}

A_{24}

A_{33}

A_{34}

A_{44}



$S \to A_{04}$

A_{00}

A_{01}

A_{02}

A_{03}

$A_{04} \to A_{13}$

A_{11}

A_{12}

$A_{13} \to A_{22} \mid 0A_{13}1$

A_{14}

A_{22}

A_{23}

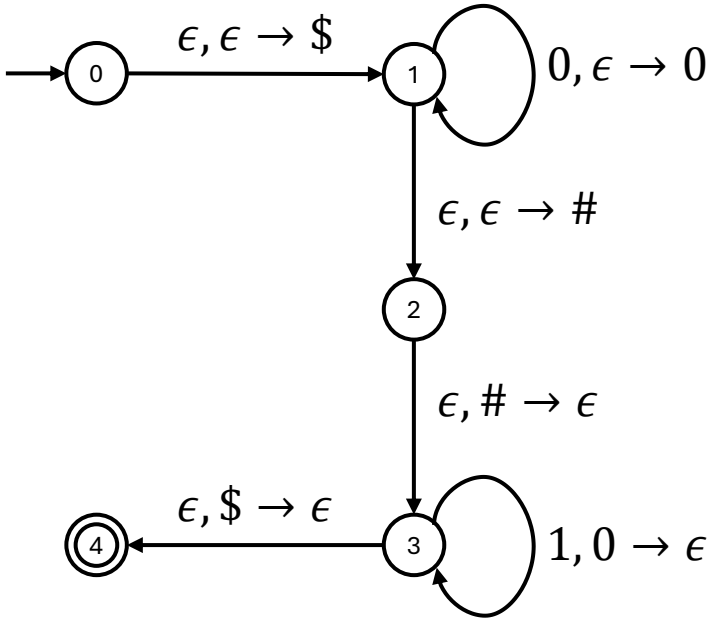
A_{24}

A_{33}

A_{34}

A_{44}

For each $p \in Q$ add the rule $A_{pp} \rightarrow \epsilon$ to G.



$S \to A_{04}$

$A_{00} \to \epsilon$

A_{01}

A_{02}

A_{03}

$A_{04} \to A_{13}$

$A_{11} \to \epsilon$

A_{12}

$A_{13} \to A_{22} \mid 0A_{13}1$

A_{14}

$A_{22} \to \epsilon$

A_{23}

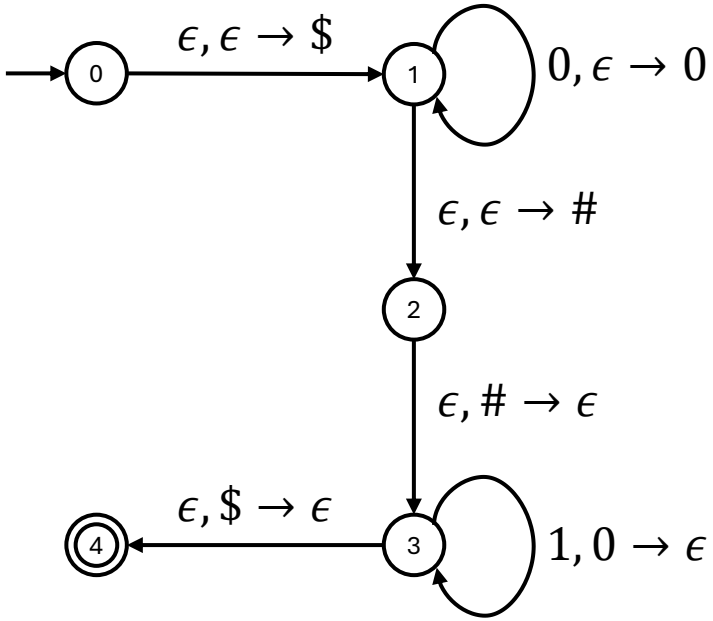
A_{24}

$A_{33} \to \epsilon$

A_{34}

$A_{44} \to \epsilon$

For each $p, q, r \in Q$ add the rule $A_{pq} \rightarrow A_{pr}A_{rq}$ to G.



$S \to A_{04}$

$A_{00} \to \epsilon$

A_{01}

$A_{02} \to A_{01}A_{12}$

$A_{03} \to A_{01}A_{13} \mid A_{12}A_{23}$

$A_{04} \to A_{13} \mid A_{01}A_{14} \mid A_{02}A_{24} \mid A_{03}A_{34}$

$A_{11} \to \epsilon$

A_{12}

$A_{13} \to A_{22} \mid 0A_{13}1$

$A_{14} \to A_{12}A_{24} \mid A_{13}A_{34}$

$A_{22} \to \epsilon$

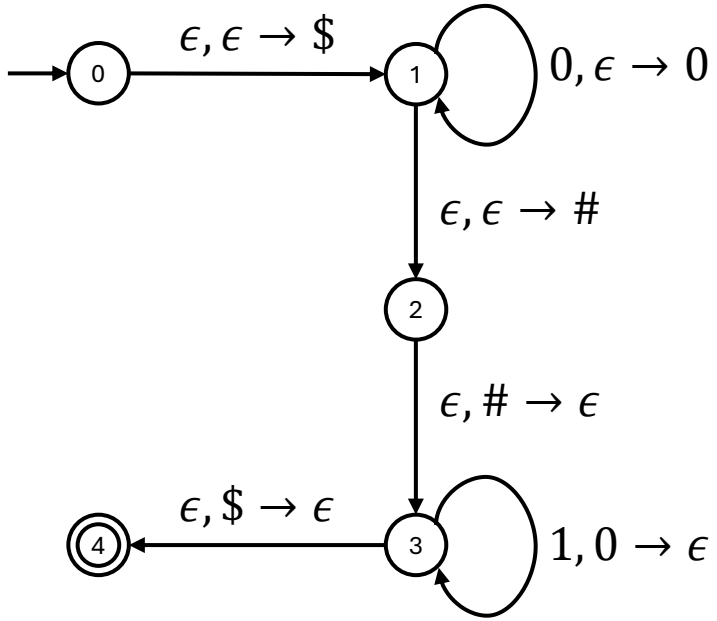
A_{23}

$A_{24} \to A_{23}A_{34}$

$A_{33} \to \epsilon$

A_{34}

$A_{44} \to \epsilon$



$S \to A_{04}$

$A_{00} \to \epsilon$

A_{01}

$A_{02} \to A_{01}A_{12}$

$A_{03} \to A_{01}A_{13} \mid A_{12}A_{23}$

$A_{04} \to A_{13} \mid A_{01}A_{14} \mid A_{02}A_{24} \mid A_{03}A_{34}$

$A_{11} \to \epsilon$

A_{12}

$A_{13} \to A_{22} \mid 0A_{13}1$

$A_{14} \to A_{12}A_{24} \mid A_{13}A_{34}$

$A_{22} \to \epsilon$

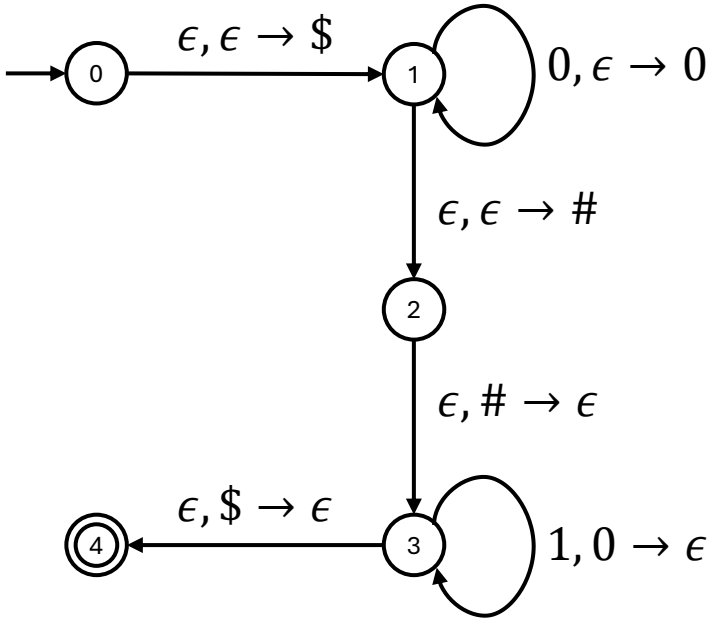
A_{23}

$A_{24} \to A_{23}A_{34}$

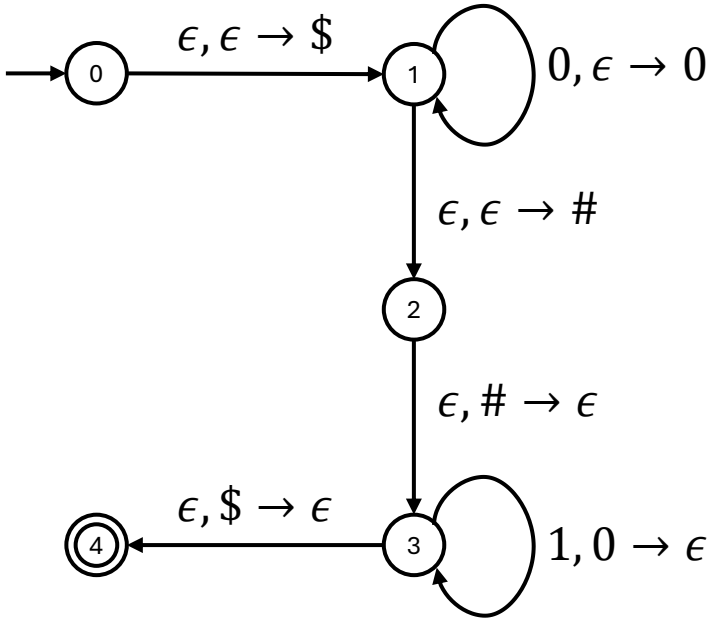
$A_{33} \to \epsilon$

A_{34}

$A_{44} \to \epsilon$

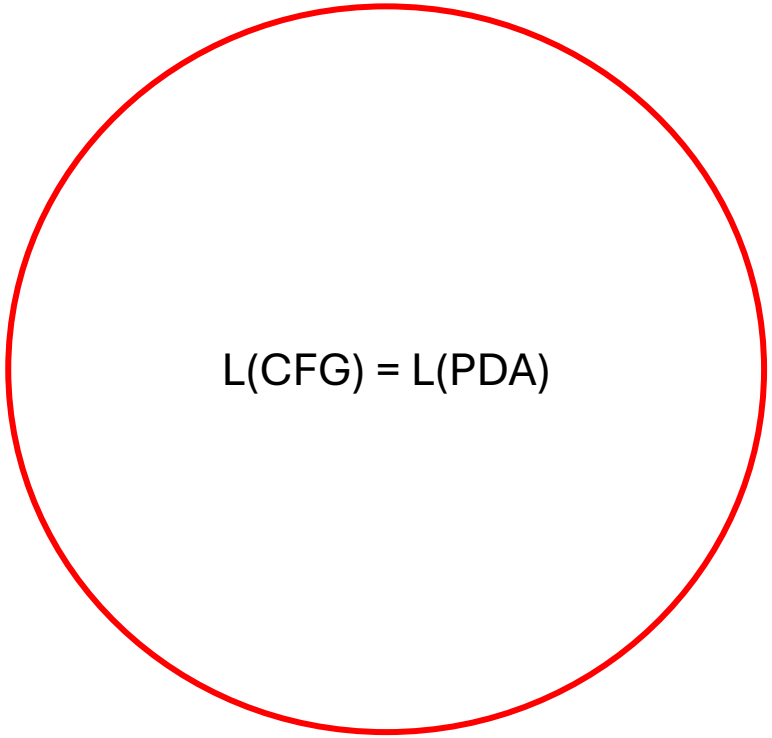


$$S \to A_{04}$$
$$A_{04} \to A_{13}$$
$$A_{13} \to A_{22} \mid 0A_{13}1$$
$$A_{22} \to \epsilon$$



$$A_{13} \to A_{22} \mid 0A_{13}1$$

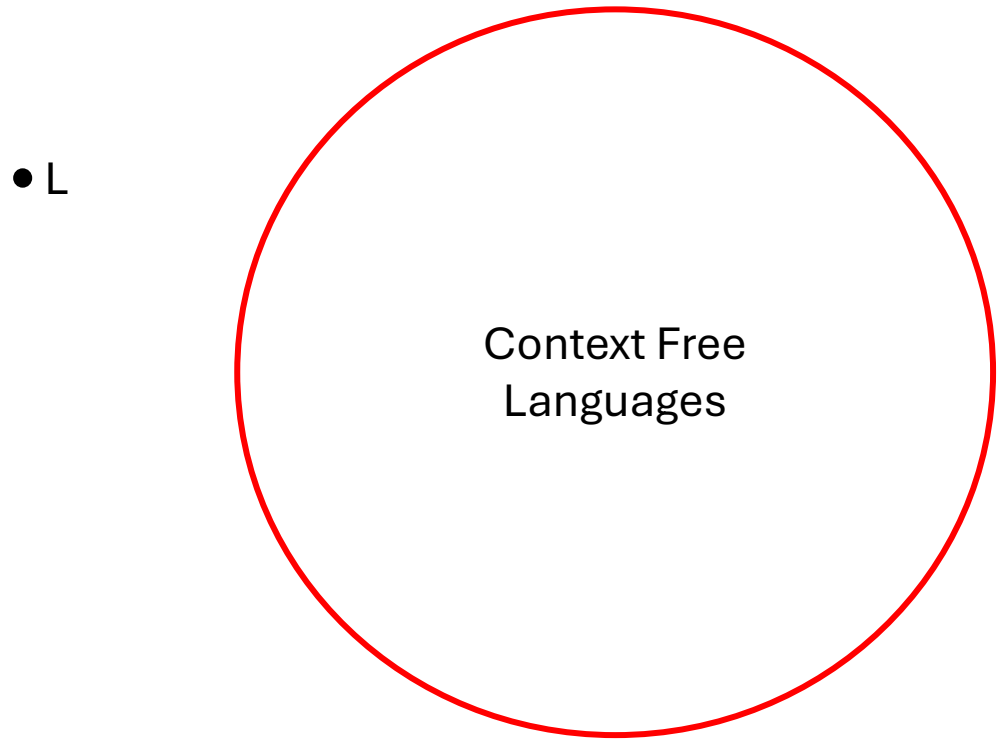
$$A_{22} \to \epsilon$$



CFGs and PDAs have equal power. Both can be used to define the class of context free languages.

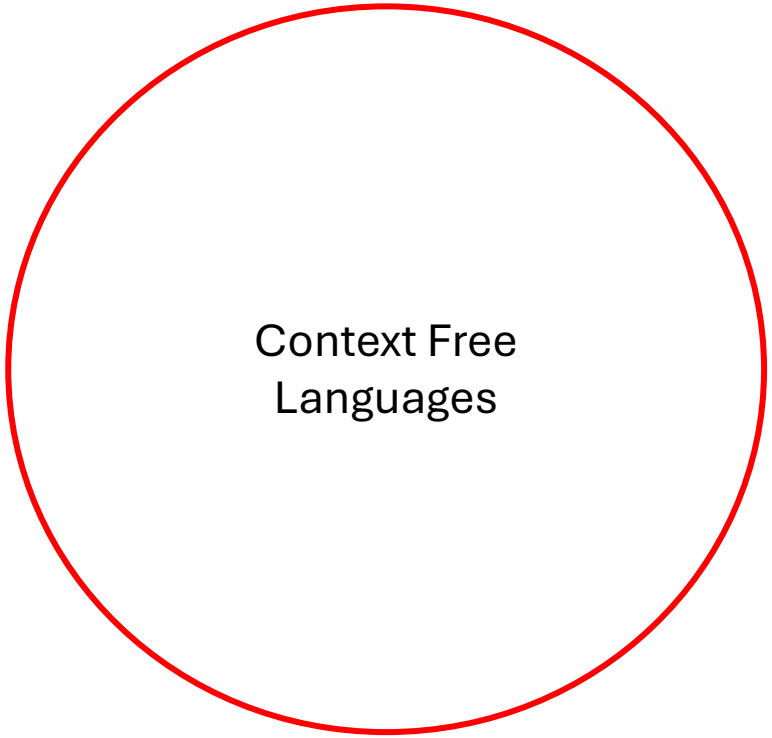
The Theory of Computing

Non-Context Free Languages



There are languages outside the context free languages.

- L



For instance, $L = \{0^n 1^n 0^n \mid n \geq 0\}$.

$$L = \{0^n 1^n 0^n \mid n \geq 0\}$$

Using the stack of a PDA to keep track of symbols will only allow us to ensure two blocks are the same, not three.

$$L = \{0^n 1^n 0^n \mid n \geq 0\}$$

Once again, this argument does not constitute a proof that this language is not context free.

To prove languages are not context free we use a theorem called the pumping lemma.

The pumping lemma for context free languages works very similarly to the regular language version, except there are now two parts we can pump.

Pumping Lemma: If L is a context free language, then there exists a number $p \geq 0$ (the pumping length) such that if $w \in L$ and $|w| \geq p$ then w can be divided into five pieces $w = uvxyz$ satisfying these three conditions:

1. For each $i \geq 0$, $uv^i xy^i z \in L$.
2. $|vy| > 0$
3. $|vxy| \leq p$

Pumping Lemma: If L is a context free language, then there exists a number $p \geq 0$ (the pumping length) such that if $w \in L$ and $|w| \geq p$ then w can be divided into five pieces $w = uvxyz$ satisfying these three conditions:

1. For each $i \geq 0, uv^i xy^i z \in L$.
2. $|vy| > 0$
3. $|vxy| \leq p$

In this version of the lemma there are five parts and both v and y can be pumped. These correspond to components processed while loading and unloading the stack of a PDA.

Pumping Lemma: If L is a context free language, then there exists a number $p \geq 0$ (the pumping length) such that if $w \in L$ and $|w| \geq p$ then w can be divided into five pieces $w = uvxyz$ satisfying these three conditions:

1. For each $i \geq 0, uv^i xy^i z \in L$.
2. $|vy| > 0$
3. $|vxy| \leq p$

In this version of the lemma the parts that can be pumped can occur anywhere in the string, not just the beginning. Their length must still be less than the finite pumping length p.

Pumping Lemma: If L is a context free language, then there exists a number $p \geq 0$ (the pumping length) such that if $w \in L$ and $|w| \geq p$ then w can be divided into five pieces $w = uvxyz$ satisfying these three conditions:

1. For each $i \geq 0, uv^i xy^i z \in L$.
2. $|vy| > 0$
3. $|vxy| \leq p$

The pieces we pump must still have some symbols in them combined.

$$L = \{0^n 1^n 0^n \mid n \geq 0\}$$

We can use the pumping lemma to show that L is not context free.

Proof Strategy: If we can show that a language does not have this property, then we can show that it is not context free.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof: Assume that L is a context free language so the pumping lemma for context free languages must hold.

(We will seek a contradiction to show that this assumption is false.)

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): If the pumping lemma holds, then any string longer than the pumping length p will have the pumping properties.

For instance, the string $s = 0^p 1^p 0^p$ is, by design, longer than the pumping length p, so s will have the pumping properties.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): By the pumping properties $s = 0^p 1^p 0^p$ can be split into five parts $s = uvxyz$ such that:

1. $|vxy| \leq p$
2. $|vy| > 0$
3. For each $i \geq 0$, $uv^i xy^i z \in L$.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): By the pumping properties $s = 0^p 1^p 0^p$ can be split into five parts $s = uvxyz$ such that:

1. $|vxy| \leq p$ means vxy can span at most two consecutive blocks of symbols.
2. $|vy| > 0$
3. For each $i \geq 0, uv^i xy^i z \in L$.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): By the pumping properties $s = 0^p 1^p 0^p$ can be split into five parts $s = uvxyz$ such that:

1. $|vxy| \leq p$ means vxy can span at most two consecutive blocks of symbols.
2. $|vy| > 0$ means v or y must have at least one symbol.
3. For each $i \geq 0, uv^i xy^i z \in L$.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): By the pumping properties $s = 0^p 1^p 0^p$ can be split into five parts $s = uvxyz$ such that:

1. $|vxy| \leq p$ means vxy can span at most two consecutive blocks of symbols.
2. $|vy| > 0$ means v or y must have at least one symbol.
3. For each $i \geq 0$, $uv^i xy^i z \in L$.

Let's pick $i = 0$, then uxz will be missing symbols from at most two of the three blocks of symbols. This means the three blocks will be of different sizes. So $uxz \notin L$ and this property doesn't hold.

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof (continued): Since the pumping property did not hold for our selected string s, we've shown that the pumping lemma doesn't hold for L.

This contradicts our assumption that L was a context free language. So, L must be not context free. □

Theorem: $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not context free.

Proof: Assume that L is a context free language so the pumping lemma for context free languages must hold. By the pumping properties the string $s = 0^p 1^p 0^p$ can be split into five parts $s = uvxyz$ such that:

1. $|vxy| \leq p$ means vxy can span at most two consecutive blocks of symbols.
2. $|vy| > 0$ means v or y must have at least one symbol.
3. For each $i \geq 0$, $uv^i xy^i z \in L$.

Let's pick $i = 0$, then uxz will be missing symbols from at most two of the three blocks of symbols. This means the three blocks will be of different sizes. So $uxz \notin L$ and this property doesn't hold.

This contradicts our assumption that L was a context free language. So, L must be not context free. □

Exercises

$L_0 = \{ww \mid w \in \{0,1\}^*\}$

$L_1 = \left\{0^n 1 0^{n^2} \mid n \geq 0\right\}$

$L_2 = \{0^n 1^m 0^l \mid l > m > n \geq 0\}$

$L_3 = \{0^n 1^n 0^m 1^m \mid m > n \geq 0\}$